WKVM: Treating Model State as a First-Class Serving Object

State-Native Inference for Recurrent and Hybrid Language Models

xiaol

Technical Report — August 2026

Abstract

Contemporary language-model servers are organized around a key–value (KV) cache whose size grows with the retained token history. Recurrent and linear attention models expose different serving physics: each request carries a fixed-size execution state, independent of context length. We present WKVM, a prototype that makes this state the primary allocation and lifecycle object. The implemented native RWKV-7 path combines typed state families, all-or-nothing slot admission, a continuous-batching scheduler over computed-token gaps, dense GPU state banks, and named versioned snapshots that can be restored, forked, persisted, and mutated with lineage. A separate Gemma guest path retains a bounded set of actual KV spans selected by value-vector routing; it is approximate and is not an RWKV recurrence.

We report the evidence with its limitations. On one RTX 4090, a single-run RWKV-7 1.5B steady-state experiment reached 8,077 tokens/s at batch 256 with a 12.19 MiB state per request. A separate 191M state-store demonstration created 2,000 snapshots at 2.29 MiB each, measured 8.2 ms median warm restore and 8.6 ms median immediately reloaded disk-backed restore, and reproduced 16 of 16 interrupted greedy continuations exactly. For bounded Gemma serving, synthetic recall averaged 0.90 versus 0.95 for full KV over 8K–32K contexts. In three clean A800 runs at batch 64 and 16K input tokens, WKVM used a reported 28.4 GiB whole-GPU peak and delivered 4.47× the comparable vLLM decode rate, but only 0.79× its end-to-end output throughput because prefill dominated. These results support fixed resident state and explicit state lifecycle management; they do not support quality equivalence or a universal speedup claim.

1 Introduction

High-throughput language-model serving has been shaped by autoregressive Transformers. Systems such as Orca, vLLM, and SGLang schedule token-level work continuously and manage large per-request KV histories through iteration-level batching, paging, and prefix reuse [1, 2, 3]. This design is a good match for full attention, where the history is both large and naturally indexed by a token prefix. It is not the only useful model-state regime. RWKV, Mamba, Gated DeltaNet, and related recurrent or linear-attention models compress history into a fixed-size state updated at every step [4, 5, 6, 7].

The distinction is more than a memory optimization. A compact recurrent state can be treated as a session object: it can be parked, restored, copied for a branch, exported, or deliberately transformed. A token-prefix cache usually assumes that the stored object is derived from a prefix, $S = f(x_{1:t})$. Controlled mutation instead produces $S' = g(S)$, whose identity and provenance cannot be expressed by the token prefix alone. Supporting that operation does not make prefix caching impossible; it requires a separate handle, version, and lineage layer.

WKVM—"WKV plus KVM"—explores this state-native design. Its current implementation contains two related but distinct systems:

1. a native pure-RWKV-7 execution path whose primary allocation unit is a fixed slot for each state family; and
2. a Gemma-4 guest path that bounds memory by retaining and routing actual KV spans. This second path is a training-free approximation, not a recurrent matrix-state implementation.

This separation matters. Earlier project design notes discuss broader GDN, Mamba, and hybrid recurrent banks, but the present native loader rejects hybrid RWKV checkpoints, and those broader backends are future work. The report therefore makes four narrower contributions:

- **A slot-native substrate.** Models declare typed fixed-footprint state families. Admission allocates one slot in every family atomically, and the scheduler advances a single computed-token invariant for prefill, decode, and resume.
- **An explicit state lifecycle.** Named, versioned records bind state bytes to token/accounting metadata and lineage. Pinned-host and safetensors-backed tiers support snapshot, restore, fork, persistence, and registered mutations in the prototype.
- **A bounded transformer case study.** Gemma routed-span mode combines an exact recent ring with content-routed retained spans and an application-level parent-token contract for safe multi-turn reuse.
- **A claim-audited evaluation.** We retain negative results, distinguish repeated from single-run evidence, and separate exact RWKV semantics from approximate Gemma comparisons.

The central conclusion is correspondingly modest: treating compact model state as a first-class object simplifies capacity accounting and enables lifecycle operations that are awkward under prefix identity alone. It does not remove prefill cost, state bandwidth, model-quality limits, or the need for mature kernels.

2 Background and Scope

2.1 KV histories and recurrent state

For a Transformer with retained length L, the KV footprint is approximately

$$M_{\mathrm{KV}}(L) = 2Lb \sum_{\ell \in \mathcal{A}} h_\ell^{\mathrm{kv}} d_\ell, \tag{1}$$

where b is bytes per element and $\mathcal{A}$ is the set of attention layers. Sliding-window and KV-sharing layers change the coefficient, but full attention remains length-dependent. PagedAttention reduces fragmentation and enables sharing without changing this semantic scaling [2].

A recurrent layer instead carries an update state

$$S_t = F(S_{t-1}, x_t), \qquad y_t = G(S_t, x_t), \tag{2}$$

whose storage is fixed for a given model. In the implemented RWKV-7 path, the state contains an FP32 matrix-valued WKV component and two model-dtype token-shift vectors per layer. The serving footprint is therefore

$$M_{\mathrm{state}} = \sum_{j=1}^{J} m_j, \tag{3}$$

where m_j is the fixed footprint of state family j; it does not depend on L.

Constant context scaling does not imply negligible cost. Linear-attention states can be large enough that reading and writing them every token becomes a bandwidth bottleneck. KVBuffer directly targets this issue by buffering recent K/V and amortizing recurrent-state updates [8]. WKVM's slot and lifecycle abstractions are complementary to such kernel-level update policies.

2.2 Identity, reuse, and lifecycle

PagedAttention primarily addresses physical allocation; RadixAttention adds prefix-indexed reuse [2, 3]. Marconi checkpoints recurrent-layer state for hybrid-model prefix caching [9]. Pensieve and other stateful-session systems retain and tier multi-turn Transformer KV histories [10]. These systems establish that state reuse and session persistence are not new by themselves.

WKVM focuses on a narrower identity boundary. A prefix-derived checkpoint is immutable evidence of $f(x_{1:t})$. A first-class recurrent-state handle may also represent a controlled transform, merge, or decay whose parent and rule must be recorded independently of the visible token sequence. The prototype does not establish that arbitrary mutations preserve language-model quality; it supplies the mechanism and provenance needed to study them.

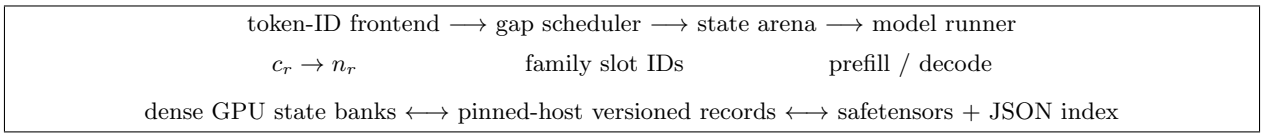

Figure 1: Implemented native RWKV-7 data path. The arena owns integer slot identities, the GPU bank owns tensors, and the optional store owns immutable snapshot records. Transfers are synchronous in the current prototype.

2.3 Goals and non-goals

The implemented goals are exact slot-count admission, continuous arrivals, fixed-footprint recurrent state ownership, explicit hibernate/resume, and a bounded-memory Gemma compatibility path. The current system is not a general RWKV/GDN/Mamba server, a drop-in replacement for the complete OpenAI API, a distributed migration service, or a quality-equivalent replacement for full KV Gemma. CUDA-graph results in this report come from a benchmark wrapper that captures the model forward for fixed batch buckets; the core RWKV runner still performs scheduler work and state gather/scatter eagerly.

3 State-Native Architecture

3.1 Typed state families and exact admission

A model exposes a `ModelStateSpec` containing one or more `StateFamilySpec` records. Each family declares a name, a byte footprint per request, and the layers that use it. The RWKV-7 implementation declares a `wkv` family and a `shift` family. The arena preallocates integer slot IDs $1, \ldots, N$ in every family; slot zero is reserved as a padded write target for static batch shapes.

For request r, allocation returns

$$A(r) = \{s_{r,1}, \ldots, s_{r,J}\}, \tag{4}$$

with one slot from every family. Admission is possible exactly when

$$\min_j |\mathcal{F}_j| > 0, \tag{5}$$

where $\mathcal{F}_j$ is the free set for family j. This removes token-block fragmentation and watermark reasoning from recurrent-state admission. It trades that simplicity for static provisioning: unused bytes inside a family slot cannot be lent to another footprint class.

The tensor owner is separate from the allocator. `RWKV7StateBank` materializes layer-major arrays so that a layer's active batch is gathered with one indexed selection and committed with one indexed copy. Python request objects carry only slot IDs, not state tensors.

3.2 One computed-token scheduling invariant

Each request stores prompt tokens, sampled output tokens, and a single counter c_r for tokens whose state contribution has been computed. Its current target is

$$n_r = |x_r| + |y_r|, \qquad g_r = n_r - c_r. \tag{6}$$

A request with $g_r = 1$ is in steady-state decode. A request with a larger gap has prefill or continuation work. A restored request simply begins with $c_r > 0$ and a nonempty gap; it does not require a separate scheduler phase.

At every step, the scheduler first advances running requests under a global token budget, then admits waiting requests while slots and running capacity remain. A per-request cap prevents a long prompt from consuming the entire step. Completed requests release their slots during the same update, allowing new arrivals to enter on the next step. A completion-prefill lane is also implemented for workload profiles that favor finishing a bounded cohort of long prompts while preserving decode priority.

3.3 RWKV execution and graph buckets

The native loader accepts FLA-format pure RWKV-7 checkpoints. Prefill seeds an FLA cache from arena slots, executes chunks, and writes the resulting recurrent and shift states back after each chunk. Decode gathers a changing set of slot rows, performs one recurrent step for the batch, and scatters the state back. The reference FLA kernels provide chunked prefill and fused recurrent decode; WKVM owns scheduling and persistent state placement [11].

The M2 benchmark adds one CUDA graph for the model forward at each measured batch size. Static graph input and output buffers are replayed when the batch shape is unchanged. State gather/scatter, sampling, and scheduler Python remain eager, so these results should not be described as an integrated whole-engine graph implementation.

3.4 Versioned durable-state records

An optional `StateStore` snapshots a live GPU slot into pinned host memory. A record can be summarized as

$$H = (\text{name}, \text{version}, \phi, c, \tau, \text{parent}, \text{rule}, \theta), \tag{7}$$

where ϕ is the current state-layout fingerprint, c is the computed-token count, τ is the known token list, and θ stores rule parameters. Handles have the form `name@version`; versions are append-only and records are immutable after creation.

The prototype exposes the following operations:

- **snapshot/hibernate**: copy state from the GPU bank to pinned host memory, optionally releasing the live slot;
- **restore**: allocate a slot, transfer the saved tensors to it, and submit a request whose c_r starts at the saved count;
- **fork**: create a new metadata record sharing immutable warm tensors until a transformed child is materialized;
- **persist**: write contiguous tensors to one safetensors file and update an atomically replaced JSON index; and
- **mutate**: clone the parent tensors, apply a registered rule such as decay or merge, and record parent/rule/parameters in a new version.

The present fingerprint hashes shapes and dtype, not model weights or a full checkpoint manifest. The store is synchronous and assumes a single-threaded engine. Warm-only snapshots do not survive process death; only explicitly persisted records can be rebuilt from the index. These constraints are important boundaries for any production interpretation.

4 Bounded Gemma Guest Mode

4.1 Actual retained-KV design

The Gemma guest is a separate execution engine. It constructs the text model from checkpoint tensors and includes custom scheduling, token-pool, and Triton decode paths, but it still uses Transformer configuration and tokenizer utilities at the server boundary. More importantly, its bounded memory is made of retained *actual keys and values*; it is not the RWKV-7 matrix state used by the native path.

For each supported full-attention layer, a request keeps an exact sink, an exact recent ring, a pending eviction region, and a fixed number of routed span slots. Evicted text is split at punctuation with fallbacks, a leader layer selects a span using its most novel value-vector feature, and online cosine routing assigns it to a persistent slot. Within a slot, farthest-point diversity retention prunes spans under a token budget. The commonly evaluated configuration uses sink 16, ring 1,024, pending 512, 64 routed slots, and a 144-token limit per slot. Accounting in the evidence bundle bounds retained material at approximately 10,831 tokens per row for that configuration.

This is a fixed-capacity compression policy. Recent dependencies inside the ring are exact; information outside it can be discarded, coalesced, or routed to a competing slot. It therefore belongs with bounded/streaming attention methods such as StreamingLLM and KV-cache compression, not with exact Transformer serving [12, 13].

Table 1: Selected native RWKV-7 steady-state results on one RTX 4090. Graphs capture the fixed-shape model forward, not the complete engine step.

Model	State/slot	Batch	Eager tok/s	Graphed tok/s	Peak VRAM
1.5B	12.19 MiB	1	67	195	2.96 GiB
1.5B	12.19 MiB	64	3,592	5,274	5.92 GiB
1.5B	12.19 MiB	256	7,913	8,077	19.02 GiB
191M	2.29 MiB	1	132	680	0.45 GiB
191M	2.29 MiB	256	26,707	38,856	2.94 GiB

4.2 Parent-token session contract

The Open WebUI integration retains a parked engine session after a turn and reuses it only when a provider-specific parent-token contract validates. The binding includes the model, user, chat, current and parent message IDs, exact visible parent history, and a digest of raw retained token IDs. This matters because decoding generated tokens to text and re-tokenizing is not always an identity operation. Edits, branches, stale parents, changed filters, or missing metadata cause a safe fresh prefill rather than silent reuse.

The HTTP surface is deliberately narrow: text-only greedy chat, one choice, and no tools, images, log probabilities, or arbitrary stop sequences. The regular server binds to loopback and has no built-in API-key enforcement.

5 Evaluation

5.1 Questions and evidence policy

We ask four questions:

Q1. Does the native RWKV path preserve a fixed per-session footprint while supporting large decode batches?

Q2. Do lifecycle operations restore exact continuations at useful latency?

Q3. What recall is lost by bounded routed-span Gemma memory?

Q4. Do bounded resident sessions translate into end-to-end serving wins?

The experiments were produced during rapid development between July 3 and July 23, 2026. We distinguish repeated, clean campaigns from descriptive single-run checkpoints. Cross-engine Gemma rows also compare different semantics: WKVM uses `routed_span_approximate`, while vLLM and SGLang use full KV. Such ratios characterize these concrete systems and workloads; they are not quality-normalized engine rankings.

5.2 Q1: Native RWKV decode

Table 1 transcribes the M2 steady-state benchmark on one RTX 4090. Weights are BF16 and WKV state is FP32. Every timed token passes through the engine step, but each cell is one 64-step timing after eight warmup steps; raw per-repeat data and confidence intervals are unavailable.

The result demonstrates the intended footprint and batchability. Graph benefit is largest at batch one (2.91× for 1.5B) and falls to 1.02× at batch 256, where model work dominates. The comparison does not establish kernel leadership: the runner uses generic FLA kernels and eager state copies, and the experiment includes no matched incumbent serving path for the same checkpoint.

5.3 Q2: State lifecycle

The M3 demonstration uses RWKV-7 191M on the same GPU. It creates 2,000 sessions through continuous batches of 64, snapshots each to pinned host memory, and persists 500 records. Table 2 reports descriptive single-run results.

The original artifact also reports tail percentiles, but the benchmark's index-based percentile calculation is not a statistically sound p99 estimate for these sample counts, so we exclude those claims. The exactness check is

Table 2: State-store demonstration. "Disk-backed reload" was measured immediately after persistence without flushing the operating-system page cache; it is not a cold-device benchmark.

Operation	Samples	Reported result	Interpretation
Pinned-host restore to next token	224	median 8.2 ms	synchronous H2D restore
Disk-backed immediate reload	76	median 8.6 ms	page cache may dominate
Interrupted/resumed exactness	16	16/16 identical	greedy, batch-one twins
Fork creation	64	544.8 ms total	8.5 ms/fork; immutable bytes shared
Restart recovery	1 session	24-token match	explicitly persisted record
Decay mutation	1 lineage	continuation changed	mechanism demo, not quality test

Table 3: Synthetic recall means for Gemma bounded-memory variants.

Mode	8K	16K	32K	Overall
Full KV	0.97	0.95	0.94	0.95
Sink + recent ring	0.18	0.00	0.00	0.06
Temporal segment bank	0.51	0.39	0.44	0.45
Value-routed slots	0.77	0.71	0.90	0.79
Span-atomic value routing	0.89	0.91	0.90	0.90

more informative: it shows that, for the tested greedy path and state layout, snapshot transfer does not change continuation tokens. It does not test cross-checkpoint rejection because the current fingerprint omits weight identity.

5.4 Q3: Routed-span recall

The routed-span quality grid evaluates Gemma-4-E4B-it [14] at 8,192, 16,384, and 32,768 tokens on three synthetic tasks: one needle, one of eight keyed facts, and aggregation of six items. Generation is greedy and substring-scored. Needle and multi-key cells average three seeds; aggregation uses one seed. Table 3 reports means over 15 task/depth cells per context.

Span-atomic routing improves the interference-heavy multi-key task from 0.51 to 0.89 overall by keeping fact spans together. It slightly reduces the scattered aggregation score (0.86 to 0.81), and its 32K retained memory is reported at roughly 88 MiB per slot versus 36 MiB for the ring-only mode. The grid is evidence for this synthetic retrieval mechanism, not general language quality. Historical natural-document NLL increases outside the exact window, and native natural-document NLL has not been rerun.

5.5 Q4: Serving comparisons

The strongest repeated comparison is a clean three-run campaign on one NVIDIA A800-SXM4-80GB. All engines receive 64 uniform 16,384-token synthetic prompts and generate 32 fixed-length greedy tokens. Results are medians, with min–max ranges in the source artifact.

Against vLLM, WKVM reaches 4.467× the comparable decode rate but only 0.793× end-to-end throughput. Its cohort prefill wall is 84.8 s versus 73.6 s for vLLM, and prefill dominates a workload with only 32 output tokens. Against SGLang, WKVM is 1.404× on end-to-end output throughput; SGLang's decode estimate is excluded because it is based on separate-run subtraction. This campaign is the clearest counterexample to a universal speedup claim.

An RTX 4090 B16 audit gives a complementary context-scaling result. At 16K input and 128 output tokens, three WKVM runs average 64.769 E2E tokens/s, while one controlled vLLM run reaches 64.713—an E2E tie. Comparable decode is 256.467 versus 67.091 tokens/s, and the mean whole-GPU engine delta is 16.871 versus 18.405 GiB. When context doubles to 32K, WKVM's single-run decode remains 253.746 tokens/s and memory increases by only 0.055 GiB, while prefill roughly doubles. The archived 32K incumbent rows use a different idle GPU baseline, so their E2E ratios remain exploratory.

Finally, an application-level Open WebUI checkpoint drives 32 persisted conversations through eight synchronized turns. It completes 256 requests at 345.097 output tokens/s with 224/224 eligible continuations reused,

Table 4: Repeated A800 B64, 16K-input, 32-output campaign. Memory is the reported whole-GPU peak and is affected by each incumbent's configured pool fraction; it is not minimum required memory.

Engine	Semantics	E2E tok/s	Decode tok/s	Peak used
WKVM	routed-span approx.	21.886	121.612	28.438 GiB
vLLM 0.25.1	full KV	27.608	27.227	74.743 GiB
SGLang 0.5.15	full KV	15.587	excluded	69.930 GiB

versus 160.322 for the tested vLLM profile and 65.055 for the tested SGLang profile. This 2.153×/5.305× result is a single cross-run checkpoint with engine-dependent generated histories, approximate-versus-full semantics, and external raw request artifacts. It demonstrates the parent-token lifecycle path; it is not a repeated publication envelope.

6 Discussion

6.1 What the abstraction buys

For a fixed-footprint recurrent model, slot-native scheduling makes admission an exact count rather than a token-block packing problem. Dense state banks also make request composition an index-selection problem, which is compatible with stable batch buckets. Most importantly, a named state handle separates session identity from a token prefix. Forking, explicit persistence, and provenance-bearing mutation can therefore be API operations instead of hidden cache behavior.

This design complements rather than replaces prior work. Marconi decides when prefix-derived recurrent checkpoints are worth caching; KVBuffer decides when to flush recurrent state for better IO efficiency; Pensieve tiers attention KV for multi-turn sessions [9, 8, 10]. A future mature runtime could combine all three ideas: typed slot allocation, checkpoint-aware prefix reuse, and buffered state updates.

6.2 Where fixed state does not help

Fixed resident memory does not make prompt computation constant-time. Both the A800 and 4090 campaigns expose prefill as the principal end-to-end bottleneck. Recurrent state itself can also be bandwidth-heavy, as the diminishing graph benefit at high batch and KVBuffer's analysis emphasize. At short context, low concurrency, or short sessions, a mature full-KV engine may simply be faster.

The Gemma guest makes an additional trade: it bounds memory by discarding or coalescing history. Its recall score is workload-dependent, and exact full-KV systems retain a semantic advantage. Performance comparisons must therefore publish the semantics column, not only a throughput ratio.

6.3 Mutable state as a research capability

The built-in decay and merge rules demonstrate lineage, not validated memory editing. A transformed recurrent state may be useful for consolidation, forgetting, branching agents, or post-training experiments, but it may also produce arbitrary behavior. Necessary next steps include checkpoint-bound fingerprints, rule-specific invariants, reference recomputation where available, downstream quality evaluation, and access control for state capabilities.

7 Related Work

Transformer serving. Orca introduced iteration-level scheduling for generative models [1]. vLLM's PagedAttention reduces KV fragmentation and supports sharing, while SGLang's RadixAttention indexes reusable KV by token-prefix paths [2, 3]. Llumnix studies live migration and scheduling across LLM instances [15]. Pensieve retains and tiers multi-turn KV state [10]. These systems motivate WKVM's lifecycle vocabulary but operate primarily on attention KV rather than opaque mutable recurrent state.

Recurrent and hybrid serving. RWKV, Mamba, and Gated DeltaNet establish fixed-state sequence models and parallel/recurrent computation forms [4, 5, 6, 7]. Marconi caches prefix-derived recurrent checkpoints in hybrid LLMs [9]; KVBuffer changes the IO schedule for linear state updates [8]; STree studies speculative tree verification for hybrid state space models [16]. WKVM differs in emphasis: the recurrent state is an explicit versioned lifecycle object and the allocator's primary unit. It does not claim the first recurrent-state server or cache.

Bounded and growing memory. StreamingLLM preserves attention sinks and a recent window for constant-memory streaming, while SnapKV compresses prompt KV based on attention-derived importance [12, 13]. Memory Caching stores recurrent hidden checkpoints and aggregates selected memories to extend RNN capacity [17]. Gemma routed-span belongs to this approximate-memory family: it retains selected actual KV spans under a fixed budget. The native RWKV state-store contribution is logically separate.

8 Limitations and Threats to Validity

- **Prototype scope.** Native execution currently supports pure RWKV-7 FLA-format checkpoints. General Mamba, GDN, and hybrid state families are design targets, not implemented backends.
- **Rapid source evolution.** Results span multiple commits over three weeks. The artifact appendix freezes the report and file hashes, but a single current binary did not produce every table.
- **Limited native evaluation.** RWKV throughput and lifecycle experiments are single-run on one RTX 4090 and small 191M/1.5B checkpoints. They lack uncertainty estimates, energy measurements, and matched engines on identical RWKV weights. The historical M3 log also warns that the FLA path still needs an official-RWKV implementation cross-check.
- **Store methodology.** The disk-backed latency did not evict the OS page cache; reported p99 calculations are excluded; the fingerprint binds layout rather than checkpoint weights; copies synchronize; and the store is single-threaded.
- **Quality coverage.** Routed-span quality uses small synthetic, substring-scored grids. Native natural-document NLL and standard long-context benchmarks are missing. Approximate and full-KV Gemma are not semantically equivalent.
- **Comparison statistics.** Only the A800 cell has three clean runs for every engine. The 4090 controlled vLLM point, 32K cells, and Open WebUI checkpoint need interleaved repeats, randomized order, confidence intervals, thermal/clock controls, and committed request-level artifacts.
- **Memory interpretation.** Whole-GPU peaks include configured incumbent pools and any process on the selected device. They demonstrate observed residency under those launch profiles, not minimum memory required by each engine.

9 Artifact Availability and Reproducibility

Source, benchmark drivers, result summaries, and provenance bundles are available in the project repository [18]. The dependency-free allocator, scheduler, mixed-batch metadata, benchmark contract, and packaging subset passes 60/60 tests in the report-preparation environment. The full 794-test discovery run cannot be treated as a pass in that environment because 124 Torch-dependent tests error at import and 153 tests are skipped; historical GPU artifacts were not rerun for this report.

Table 5 lists the exact evidence promoted into the paper. A companion claim ledger in `paper/CLAIMS.md` records excluded claims and additional caveats.

10 Conclusion

WKVM demonstrates that a serving runtime can be organized around fixed-size model state rather than treating every architecture as a variant of a growing KV history. The implemented substrate provides exact family-slot

Table 5: Promoted artifact ledger. SHA-256 values identify the report files, not every external raw measurement referenced by those reports.

Evidence	Repository path	Report SHA-256
RWKV M2	`experiments/results/m2_engine_bench.md`	`899d26677dadcf27...e508df7`
State lifecycle M3	`experiments/results/m3_results.md`	`15d336e000c7c4a...b951f9`
Routed-span quality	`experiments/results/quality_grid.md`	`a5826b978281ecb...4f5c2`
4090 B16 audit	`experiments/results/gemma_b16_evidence_audit_20260713.md`	`b5eef0bf411ca5b...0fae46`
A800 repeats	`experiments/results/gemma_a800_reliable_20260716/report.md`	`83371ce4686212c...386776`
Open WebUI checkpoint	`experiments/results/open_webui_parent_token_b32_t8_20260723.md`	`aea5cce4adcba99...a908`

admission, a unified computed-token scheduler, dense recurrent state ownership, and versioned lifecycle handles. Its separate Gemma case study shows why bounded resident state can improve long-session decode and memory behavior, and also why prefill and approximation quality remain load-bearing constraints.

The current evidence is encouraging but not a universal performance claim. The most useful next milestone is not another isolated speedup: it is a checkpoint-bound, asynchronous state store plus interleaved repeated evaluation on a mature native recurrent or hybrid model, including standard quality and end-to-end workload curves.

A Reproduction Entry Points

The following commands correspond to the promoted artifact families. GPU commands require their documented model paths, accepted model licenses, and environment-specific dependencies.

```
# Dependency-free current-checkout validation
python -m unittest -v \
  tests.test_arena tests.test_scheduler tests.test_mixed_batch \
  tests.test_benchmark_contract tests.test_packaging

# Historical native RWKV measurements
python experiments/m2_engine_bench.py --model /path/to/rwkv7 --graph
python experiments/m3_demos.py --model /path/to/rwkv7

# Frozen benchmark runners (see each script's --help and report contract)
bash scripts/run_wkvm_phase3_4090.sh
bash scripts/run_wkvm_10x_http_a800.sh
```

The A800 repeated report records benchmark commit `234cc04867a93f1352d2a1c220c216b698f46560`, model-manifest SHA-256, source/worktree identity, GPU UUID, driver and package versions, launch commands, request counts, output fingerprints, and min/median/max aggregates. The sealed 4090 B16 bundle likewise contains copied JSON artifacts, source manifests, commands, and integrity hashes. Open WebUI raw request artifacts remain external and are therefore not promoted as a repeatable envelope.

B Submission-Scope Statement

This technical report intentionally excludes the following statements: "WKVM is 10× faster than vLLM/SGLang," full-quality equivalence for routed-span Gemma, first stateful LLM serving, first recurrent prefix caching, first linear-attention serving, production-ready live migration, and general Mamba/GDN/hybrid support. Any later revision that adds one of these claims should add a new controlled artifact and update the claim ledger before submission.

References

[1] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for Transformer-based generative models. In *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, pages 521–538, 2022.

[2] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, pages 611–626, 2023. arXiv:2309.06180.

[3] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Jeff Huang, Chuyue Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2312.07104.

[4] Bo Peng, Eric Alcaide, Quentin Anthony, Alon Albalak, Samuel Arcadinho, et al. RWKV: Reinventing RNNs for the Transformer era. In *Findings of the Association for Computational Linguistics: EMNLP*, pages 14048–14077, 2023. arXiv:2305.13048.

[5] Bo Peng, Ruichong Zhang, Daniel Goldstein, Eric Alcaide, Haowen Hou, et al. RWKV-7 "goose" with expressive dynamic state evolution. *arXiv preprint arXiv:2503.14456*, 2025.

[6] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *Conference on Language Modeling (COLM)*, 2024. arXiv:2312.00752.

[7] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving Mamba2 with delta rule. In *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2412.06464.

[8] Longwei Zou and Lin Zhong. KVBuffer: IO-aware serving for linear attention. *arXiv preprint arXiv:2605.19049*, 2026. doi:10.48550/arXiv.2605.19049.

[9] Rui Pan, Zhuang Wang, Zhen Jia, Can Karakus, Luca Zancato, Tri Dao, Yida Wang, and Ravi Netravali. Marconi: Prefix caching for the era of hybrid LLMs. In *Proceedings of Machine Learning and Systems (MLSys)*, 2025. arXiv:2411.19379; OpenReview: RUaMUu7vMX.

[10] Lingfan Yu, Jinkun Lin, and Jinyang Li. Stateful large language model serving with Pensieve. In *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys)*, pages 144–158, 2025. arXiv:2312.05516.

[11] FLA Team. Flash linear attention. `https://github.com/fla-org/flash-linear-attention`, 2026. Accessed 2026-08-02.

[12] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2309.17453.

[13] Yuhong Li, Yingbing Huang, Bowen Yang, Bharat Venkitesh, Acyr Locatelli, Hanchen Ye, Tianle Cai, Patrick Lewis, and Deming Chen. SnapKV: LLM knows what you are looking for before generation. *arXiv preprint arXiv:2404.14469*, 2024.

[14] Google DeepMind. Gemma-4-e4b-it model card. `https://huggingface.co/google/gemma-4-E4B-it`, 2026. Accessed 2026-08-02.

[15] Biao Sun, Ziming Huang, Hanyu Zhao, Wencong Xiao, Xinyi Zhang, Yong Li, and Wei Lin. Llumnix: Dynamic scheduling for large language model serving. *arXiv preprint arXiv:2406.03243*, 2024.

[16] Yangchao Wu, Zongyue Qin, Alex Wong, and Stefano Soatto. STree: Speculative tree decoding for hybrid state-space models. *arXiv preprint arXiv:2505.14969*, 2025.

[17] Ali Behrouz, Zeman Li, Yuan Deng, Peilin Zhong, Meisam Razaviyayn, and Vahab Mirrokni. Memory caching: RNNs with growing memory. *arXiv preprint arXiv:2602.24281*, 2026.

[18] xiaol. WKVM: A state-native inference prototype. `https://github.com/xiaol/wkvm`, 2026. Accessed 2026-08-02.